

PYRAX Network — Technical Whitepaper

Version 4.0 · 2026-06-30 License: Open-core — Apache-2.0 (protocol / node / SDK) · proprietary apps, wallet, NEURAX & services

Abstract

PYRAX is a from-scratch, Rust-implemented Layer-1 protocol organized around four properties most networks bolt on but PYRAX enforces as invariants in its lowest-level types: **high throughput**, **privacy by default**, **full decentralization**, and **resistance to network-level (ISP) surveillance**. Consensus is not a linear chain but a **DAG** ordered by **GhostDAG** (k-cluster blue-set selection), fed by a **TriStream** model — two independent proof-of-work families across four seal lanes plus proof-of-stake — and finalized by **BLS-aggregated proof-of-stake BFT**. Value transfer is **shielded by default** using an Orchard-style note-commitment / nullifier model with recursive zk-SNARKs that require **no trusted setup**; transparent transfers exist only as an explicit special case. The peer layer is **bootstrapless**, and node traffic (plus first-class anonymous file-transfer and media-streaming services) rides an **onion-routed Sphinx mixnet**. Execution scales outward through a **multi-VM L2** (EVM, WASM, Cairo) and a **recursive ZK-rollup L3**, and the same idle GPU/CPU capacity that mines Stream B powers **NEURAX** — a verifiable, on-chain-settled AI and compute marketplace. A zero-fee first-party miner (**Crucible**) and an autonomous operations system (**NOVA**) complete the ecosystem.

Unlike earlier revisions of this paper, **v4 describes a system that is substantially built and tested, not merely designed**. The protocol runs as **one binary across four chainspecs**; the live networks are a **faithful simulation on real primitives** — real GhostDAG, real BLS/ZK cryptography, real Merkle-Patricia state, real VMs — with a real `ConsensusMode::Production` path wired end-to-end. The open protocol (node, consensus, ZK circuits, SDK, CLI) is licensed under **Apache-2.0** so anyone can run and independently verify it; the wallet, apps, NEURAX, and hosted services are proprietary. This document specifies the architecture **as it is implemented today**, and is honest about what remains gated behind external audit before mainnet.

1. Design Goals — Five North Stars

#	Property	What it means concretely
1	DAG, not a chain	A multi-parent DAG ordered by GhostDAG. Blocks reference many tips, so honest parallel work is <i>included</i> , not orphaned — high throughput, fast confirmations.
2	Privacy by default (shielded)	The default transaction hides sender, receiver, and amount via no-trusted-setup zk-SNARKs over note commitments and nullifiers. Transparent transfers are the explicit exception.
3	100% decentralized, no boot node	Peer discovery is bootstrapless: mDNS, Kademlia DHT, peer exchange, and a signed community seed list. No project-run server on the critical path to joining.
4	Metadata privacy / ISP-resistance	Node traffic and the file/media services ride an onion Sphinx mixnet with fixed-size packets, so an on-path observer sees only uniform padded encrypted flows.
5	Open & verifiable	The protocol — node, consensus, ZK circuits, CLI, SDK — is Apache-2.0. A chain with secret rules is not trustless. Apps, wallet, NEURAX, and services are proprietary; the moat is the brand, network effects, and UX, not source secrecy.

These are constraints, not aspirations: the foundation crates encode DAG-aware and privacy-aware types so every later layer is built against them.

2. Architecture Overview

PYRAX is layered. **L1** is a GhostDAG with shielded-by-default state. **L2** provides three anchored execution environments. **L3** is a recursive ZK-rollup that settles to L1. Two cross-cutting transports — a bootstrapless libp2p mesh and an onion mixnet over it — carry all traffic; the application tier (wallet, node/miner/AI app, anonymous file drop, media streaming, NEURAX) sits on top.

Apps: Wallet (shielded+transparent) · Inferno/Ember node+miner+AI app · CLI · File Drop
· Media Streaming

| NEURAX – verifiable GPU/CPU compute + AI marketplace (on-chain escrow, 4-rung verification)

| L3 – recursive ZK rollup (plonky2; validity proofs fold to one, settle at depth 32)

| L2 – EVM (revm) · WASM (wasmtime) · Cairo (cairo-vm) – cross-VM, EIP-150 63/64, precompiles

| L1 – PYRAX GhostDAG (Rust)

• Shielded-by-default tx (Poseidon note tree depth 32 + nullifier set + plonky2 proof)

• GhostDAG ordering (blue set / k-cluster, k=16) over a multi-parent DAG

• TriStream: A(BLAKE3+SHA-256d) · B(kHeavyHash-GPU + Argon2id-CPU) · C(BLS PoS finality)

• Merkle-Patricia state (redb+LZ4) with getProof; EIP-1559 fee market

▲ Onion Sphinx MIXNET (fixed-size packets, ISP-resistant) over the P2P mesh

▲

▲ Bootstrapless libp2p (mDNS · Kademlia · peer-exchange · signed seed list · QUIC/TCP)

▲

The implementation is a Cargo workspace of single-responsibility crates (toolchain pinned to `nightly-2026-06-10`; the recursive ZK prover is **default-on** across the snark chain).

Crate	Role
pyrax-crypto	BLAKE3/SHA-256/Keccak-256, secp256k1/Ed25519, BLS12-381 (blst) with proofs of possession , ChaCha20-Poly1305 AEAD, X25519, ZK-privacy primitives.
pyrax-primitives	Multi-parent GhostDAG block types, the 6-variant Tx envelope, EIP-1559 params, SealAlgo .
pyrax-codec / pyrax-proto	Canonical deterministic binary codec + protobuf bridge (never-panic decoders).
pyrax-dag	GhostDAG store, k-cluster blue-set coloring, blue work, per-lane DAA, reachability oracle.
pyrax-consensus	TriStream + GhostDAG orchestration, DevSealer, reward/emission, ConsensusMode .
pyrax-zk	plonky2 proving: shielded circuits, L3 validity + recursion; pinned circuit digests.
pyrax-privacy	Shielded pool: Poseidon note-commitment tree (depth 32), nullifier set, viewing keys, note encryption.
pyrax-mixnet	Sphinx onion routing over Ristretto (5 hops, fixed 896-byte packets).
pyrax-stream	E2E file transfer + media streaming: chunk DAG, want/have pull, signed relay directory.
pyrax-p2p	Bootstrapless libp2p mesh + GhostDAG gossip + PeerTable (eclipse/ban control).
pyrax-storage / -state / -mempool	redb+LZ4 KV, Merkle-Patricia state + proofs, transaction pool.
pyrax-vm-{evm,wasm,cairo} / pyrax-contracts (crate)	Multi-VM L2, cross-VM runner, precompiles, native protocol contracts.
pyrax-rollup	L3 sequencer/execution (crypto core in pyrax-zk).
pyrax-job-market / -weights / -carriage / -local-runtime / -model-registry	NEURAX compute market, model CDN, activation channel, local-GPU router.
pyrax-rpc / -sdk / -node	JSON-RPC (eth_* + pyrax_*), typed client SDK, the node binary.

3. Performance & Scalability — the $\geq 500,000$ TPS Target

PYRAX targets $\geq 500,000$ **sustained transactions per second** as a **network-wide aggregate** with a high L1 floor. This is a measured engineering mandate, not a marketing figure: **no throughput number is published without a reproducible benchmark** stating the workload and the exact hardware/bandwidth. The headline metric is simple value transfers; contract-call TPS and shielded TPS are measured and reported separately.

No single trick reaches 500k; PYRAX composes several:

Lever	Mechanism
DAG parallelism	GhostDAG accepts many blocks concurrently — no single-leader bottleneck; the confirmable block rate rises with network capacity.
Parallel execution	Non-conflicting transactions execute concurrently (optimistic, Block-STM-style), deterministically re-executed on conflict.
L2 / L3 rollups	Most throughput lives in rollups; the L3 recursive ZK rollup batches enormous counts and settles one succinct proof to L1.
Recursive proof aggregation	Thousands of shielded proofs fold into one recursive proof, so a validator performs $\approx$ one verification per batch — the key that reconciles <i>private by default</i> with 500k TPS.
Data-availability sampling	Light and mobile clients verify data availability by sampling erasure-coded shares instead of replaying everything.
Hardware-accelerated proving	GPU provers (Stream-B hardware) generate shielded and rollup proofs.
Fast networking	QUIC, compact/binary block relay, transaction batching keep propagation ahead of execution.

Decentralization is preserved by **node-class profiles** — archival, full, light, mobile — so consumer-grade machines still participate; the target never silently trades away decentralization. The number is reached **progressively** and each step is gated by reproducible `pyrax-bench` figures.

4. Consensus: GhostDAG + TriStream + BLS PoS Finality

4.1 Why a DAG

A linear chain forces one parent per block, so concurrent honest blocks become orphans — wasted work that lowers effective honest hashpower. PYRAX uses a **DAG**: a block may reference many current tips. Concurrent honest blocks are absorbed, raising throughput. The cost — "which block came first" is no longer structural — is solved by GhostDAG computing a total order.

4.2 The DAG block

```
DagBlockHeader {
    parents: Vec<Hash>      // many parents; parents[0] is the SELECTED PARENT (heaviest by
blue_score, blue_work, hash)
    blue_score: u64         // size of the ordered blue set in this block's past
    blue_work:  u128        // accumulated blue work – the cross-lane fork-choice weight
    daa_score:  u64         // difficulty-adjustment score
    state_root: Hash        // Merkle-Patricia accounts  $\oplus$  note tree  $\oplus$  nullifier acc  $\oplus$  escrow  $\oplus$ 
staking  $\oplus$  gov
    transactions_root, receipts_root: Hash
    timestamp:  Timestamp
    stream:     Stream      // A | B | C
    seal_algo:  SealAlgo    // the specific lane (committed into the PoW preimage)
    seal:       Vec<u8>     // opaque PoW/PoS seal
}
```

The block hash is BLAKE3 over the deterministic canonical encoding; any change to any field (including the parent set or the seal lane) changes the hash.

4.3 GhostDAG ordering

- **Selected parent / selected-parent chain.** `parents[0]` is the heaviest-past parent; following it to genesis gives GhostDAG's "main chain."
- **Blue set / k-cluster ($k = 16$).** A mergeset block is **blue** iff adding it keeps its blue-anticone $\leq k$ (honest blocks within $\sim$ one propagation delay stay blue); a secretly-mined block accrues a large anticone against the honest blue set and is colored **red**.
- **Blue score & blue work.** `blue_score` = blues in the past; `blue_work` = Σ block difficulty of blues (saturating). The heaviest-blue-work tip wins; **no single lane's hashrate can reorder blocks from other lanes** (invariant C7).
- **Deterministic mergeset ordering.** `order_dag` emits the topological mergeset then the block; blue ahead of red, ties by hash — the canonical execution order in which roots are computed.
- **Reachability oracle.** An $O(1)$ ancestry index (differential-tested against parent-DFS) plus a bounded coloring cache; this fixed an $O(\text{chain}^3)$ merge-coloring pathology.

`validate_block` recomputes `blue_score / blue_work` (rejecting forged weight) and enforces `parents[0] == selected_parent`.

4.4 TriStream: three streams, five seal lanes

Stream	Seal lane (SealAlgo)	Algorithm	Hardware
A	Blake3Pow (1)	$\text{blake3}(\text{header}) \leq \text{target}$	ASIC / specialized
A	Sha256dPow (2)	double-SHA-256	ASIC
B	KHeavyHashPow (3)	64×64 matrix heavy-hash (one matmul + Keccak per nonce)	Commodity GPU
B	Argon2idPow (4)	Argon2id (8 MiB, t=1, p=1), memory-hard	Commodity CPU
C	PosBls (5)	BLS signature by the slot-elected proposer	Staked validators

Three streams with different hardware bases is a deliberate security choice (§4.6). The lane is a **free producer decision committed into the PoW preimage**, so a peer cannot relabel a mined block into an easier lane. **Per-lane difficulty (DAA)** samples only same-lane ancestors along the selected-parent chain, retargets $\text{avg_work} \times \text{expected/actual}$ **clamped to $[\frac{1}{4}\times, 4\times]$** , with the PoW target computed on the high 128 bits of the hash (toolchain-free). Reward is split A/B/C by thirds (each lane in a stream shares its stream's third). *Correcting earlier drafts: Stream B is **kHeavyHash + Argon2id**, not KAWPOW/RandomX/Cuckaroo.*

Consensus modes. The Seed network runs `Simulated` (rotate A/B/C on instant-seal; a production chain-id can never do this — `produces_dev_blocks()` is the honest gate). Forge/Rise/One run `Production` (real 5-lane). In Production, the node itself auto-mines the **CPU-feasible Stream A** and proposes **Stream C**; the heavy **kHeavyHash-GPU and Argon2id-CPU lanes are supplied by external miners** through `pyrax_getWork` / `pyrax_submitWork` (a CPU cannot mine kHeavyHash at difficulty).

4.5 BLS-aggregated PoS BFT finality

PoW gives probabilistic ordering; Stream C adds deterministic finality. Validators bond stake (**minimum 32 PYRX**, ~7-day unbonding) and vote on GhostDAG-ordered blocks. Votes are **BLS12-381 min-pubkey signatures (48-byte G1 keys, 96-byte G2 sigs) that aggregate** — hundreds compress to one signature. Each validator registers a **proof of possession**; `verify_aggregate` additionally subgroup- and infinity-validates every key, defeating the rogue-key attack. A block is **final once aggregate attestations cover > 2/3 of staked weight**; the head is gated to descend from the finalized hash (**finality overrides work**). Equivocation is **slashable (5% burn + 10% reporter bounty)**.

4.6 51%-resistance

Rewriting history requires a majority of **Stream-A ASIC and Stream-B GPU+CPU** hashpower *and* a staked supermajority in **Stream C** — three uncorrelated resources with different supply chains — while GhostDAG colors withheld branches red (they lose on blue work) and BLS finality makes reverting a finalized block a slashable offense.

5. Privacy: Shielded by Default

5.1 The default is private

On production networks the default `Tx` variant is `Shielded`; transparent transfers are the explicit special case, enforced at the type level. (The Seed sandbox is transparent-only for developer ergonomics.)

5.2 The Orchard-style note model

Shielded state is a set of **notes**, not balances, kept in two structures: a growing **note-commitment Merkle tree** (depth **32** → **2^{32} leaves**) whose roots are **anchors**, and a **nullifier set**. Creating a note appends its commitment; spending reveals its nullifier (added to the set) but not which commitment it corresponds to. The on-wire transaction reveals only:

```
ShieldedTransaction {
    chain_id, anchor,
    nullifiers: Vec<Hash>,    commitments: Vec<Hash>,
    value_balance: i128,      // net transparent flow; 0 for a fully-shielded transfer
    ciphertexts: Vec<Vec<u8>>, proof, binding_sig
}
```

No sender, no receiver, no per-note amount. A flat `SHIELDED_FEE = 100 base units` is burned per shielded tx (anti-DoS).

5.3 Primitives and circuit

The default `snark` backend uses **Poseidon over the Goldilocks field** ($p = 2^{64}-2^{32}+1$) for the note commitment `Poseidon(DOM_CM,...)`, the nullifier `Poseidon(DOM_NF, nk, cm)` (position-independent, one nullifier per note), and Merkle nodes; the transparent path uses BLAKE3. **Note encryption** wraps ChaCha20-Poly1305 under an X25519-ECDH shared secret with a per-note nonce. **Viewing keys** separate "see" from "spend."

The spend/output circuit (plonky2 1.0.2, default build) proves in zero knowledge: (1) each spent note's commitment is in the tree at the stated anchor; (2) each nullifier is correctly derived (spend authority) and — checked outside the proof against the set — unused; (3) value is conserved (`inputs = outputs + value_balance`, all amounts hidden, range-checked to **60 bits**); (4) each output is well-formed. **Circuit digests are pinned** (`verify_pinned_circuits` refuses to run on mismatch) — the code-level ZK-audit gate. On-chain verification is real on the apply path. Value balance is independently clamped to $\pm 2^{60}$ at the ledger even for non-snark nodes.

5.4 Shielding, deshielding, transparent transfers

Transparent transfers are the familiar secp256k1-signed `Transaction` (sender recoverable, low-s enforced). The `value_balance` field is the boundary: **shield** (positive — transparent value consumed, notes created), **deshield** (negative — notes spent, transparent value produced), **pure shielded** (zero, indistinguishable from any other). Double-spends are impossible: any repeated nullifier is rejected.

6. Execution: Multi-VM L2

L1 is deliberately lean; rich programmability lives above it. Three VMs run over a shared `StateBackend` (20-byte address, 32-byte slots), selected by **code magic bytes**: `\0asm` → WASM, `\0CAIRO` → Cairo, else EVM.

VM	Engine	Gas	Determinism / limits
EVM	revm 22.0.1	ledger-charged (revm runs <code>gas_price=0 / disable_base_fee</code> , no double-charge; BASEFEE still real)	full journal/checkpoint transactionality
WASM	wasmtime 33.0.2	fuel 1:1	NaN-canonicalized, no threads/relaxed-SIMD, 512 KiB stack; code 256 KiB, mem 16 MiB, log 1 MiB
Cairo	cairo-vm 2.5.0	steps ~1/gas (4M cap)	non-canonical felt reverts; only <code>pyrax.*</code> hints

Cross-VM. Any VM can call any other VM or a precompile synchronously. Gas forwarding follows **EIP-150 (63/64)** — `forwarded = remaining - remaining/64` (revm handles EVM internally); the call depth is capped at `MAX_CALL_DEPTH = 1024`; value moves exactly once; `STATICCALL` read-only is inherited.

EIP-1559 fee market. `BLOCK_GAS_LIMIT = 30,000,000`, gas target = limit/2, base fee moves **±12.5%/block**, `MIN_BASE_FEE = 1`. The **fee split is consensus-frozen**: base fee **25% burned / 50% PYRAX treasury / 25% DAO**; priority tip **70% producer / 20% treasury / 10% DAO**.

Precompiles (`0x...01xx`): 0x0100 BRIDGE, 0x0101 BLAKE3, 0x0102 SHA256, 0x0103 KECCAK256, 0x0104 ECRECOVER, 0x0110 CHAIN_CONTEXT (chain-id/number/timestamp), 0x0111 SHIELDED_VIEW (anchor).

Six transaction envelopes (the `Tx` enum, all implemented): **0 Shielded**, **1 Transparent**, **2 Ethereum** (raw EIP-2718 — MetaMask/Foundry drop-in), **3 Escrow** (NEURAX Lock/Refund/Release/Drip/Split), **4 Stake** (Bond/Unbond/Withdraw/Slash), **5 Gov** (Propose/Vote). The full standard `eth_*` JSON-RPC (blocks, txs, receipts, logs, filters, subscriptions, getProof, call/estimateGas, feeHistory) plus the native `pyrax_*` namespace serve all VMs (§18).

The protocol-level functions (native token, staking, bridge, ZK verifier, AI-compute escrow) are implemented **natively in the core**. The separate `pyrax-contracts` repository of Solidity/Cairo "system contracts" is a Phase-7 stub whose bodies revert and is not deployed.

7. L3 — Recursive ZK Rollup

The L3 is a validity (ZK) rollup: a sequencer batches L3 transactions, a prover generates a validity proof per batch, and — the defining feature — proofs are **recursively aggregated** (2-to-1) into a single succinct

proof an L1 verifier checks before settlement, reusing the same no-trusted-setup plonky2 stack as shielded transactions. The **cryptographic core is real**: `pyrax-zk::rollup_circuit` implements the validity circuit, genuine recursive aggregation, and an on-L1 verifier over a sparse account tree at **depth 32 (2³² accounts)** — a production-depth aggregate proves in ~18 s.

Honest status. The `pyrax-rollup` crate's sequencer/execution is built, but its `prove / aggregate / post` orchestration methods are `unimplemented!()`, and the live bridge deliberately uses a reject-all `DeferredVerifier` so the rollup root never advances until the sequencer integration and external ZK audit land. The L3 is thus *crypto-complete, go-live-deferred*.

8. Networking: Bootstrapless Decentralization

`pyrax-p2p` is built on **libp2p 0.55** (Noise + Yamux over **QUIC/TCP**, plus WebRTC) and is **bootstrapless**: mDNS (LAN), Kademlia DHT (global crawl + rendezvous), peer exchange, and a cryptographically **signed community seed list** (a data artifact anyone can host — not a server PYRAX runs). Discovery vs. operator behavior is a config flag, not infrastructure.

Blocks and shielded transactions propagate over **GossipSub v1.1** (Strict + Signed, 8 MiB max, 200 ms heartbeat, 6 chain-scoped topics) with peer scoring. Because the ledger is a DAG, sync is **tip exchange + mergeset download**. The **PeerTable** (live-wired) enforces `max_peers = 128`, ban scoring with escalating backoff, `PROTOCOL_VERSION` epoch deny-listing, and an **eclipse cap of 8 peers per /24 (v4) or /48 (v6)**.

Sync honesty: the target is the k-th-largest *fresh* peer tip (`SEAL_CORROBORATORS = 2`, freshness 20 s), so a minority of liars cannot stall the seal gate, and a node at its own tip with no fresh peer tip reports SYNCING rather than a false 100%. The wire format is binary/CBOR; decoders never panic.

9. Metadata Privacy and Anonymous Services

9.1 The Sphinx mixnet

`pyrax-mixnet` sits as a transport *under* `pyrax-p2p`: **Sphinx onion routing over Ristretto**, `MAX_HOPS = 5`, constant `PACKET_LEN = 896 bytes`, a 2²⁰ replay window, and constant-time MAC with identity-point rejection. Each relay peels exactly one layer — learning only the next hop, never the source, destination, or contents — and every packet is identical in size. *Cover-traffic policy and constructors exist; the emitter loop and bandwidth-weighted route selection are a Phase-5 residual.*

9.2 File transfer and media streaming (`pyrax-stream`, Phase 8)

Files are split into chunks (sized to fit one onion), each content-addressed by BLAKE3 and E2E-encrypted with per-chunk capability-gated keys, organized by a content-addressed manifest; availability is advertised over the DHT; transfers are resumable. Live media is segmented, encrypted, and distributed through a swarm — a decentralized CDN with no origin server. A **want/have pull protocol** and a **signed, revocable relay directory** let content-blind relays serve without seeing plaintext. **Relay accounting is a seam; paid-relay settlement is a Phase-9 no-op stub.**

10. State, Storage, and Proofs

Storage is **redb + LZ4 (pure-Rust ACID KV)** — *not* RocksDB (surviving comments are stale) — with Blocks/State/Receipts/Meta columns. State is a **Merkle-Patricia trie** (BLAKE3 nodes, secure-trie hashed keys) with sound **inclusion and exclusion** proofs, backing `eth_getProof`. The **state root** is `blake3(PYRAX_STATE_ROOT_v3 || account_trie_root || notes_root || nullifier_acc || escrow_acc || staking_commit || gov_commit)`; the account leaf folds each account's real `storage_root`. **Fast-sync** serves a root-verified `LedgerSnapshot` via `pyrax_snapshot`. The **mempool** caps 16,384 total / 256 per-sender / 4,096 shielded, ordered by contiguous nonce runs, highest-fee-head first.

11. NEURAX — Verifiable AI & Compute Marketplace

PYRAX turns the GPU/CPU hardware that mines Stream B into an **on-chain-settled compute market**. The proprietary `neurax-ai` repository is a scaffold, but the implementation is **built and green inside the Apache core** (a deliberate adopt-then-own, one-way dependency): the `pyrax-job-market`, `-weights`, `-carriage`, `-local-runtime`, and `-model-registry` crates (~6.9k LOC, ~106 tests).

Thesis: a federation of tiered specialist experts (text/image/video/audio/ML-platform/copilot) behind one agentic router. NEURAX does **no frontier pretraining** — it fine-tunes/distills open bases and adopts mature runtimes; the moat is post-training + orchestration + verification + copilot + market.

Job lifecycle (`pyrax-job-market`): `submit` → `escrow.lock` → `match` (redundantly) → `execute` (×quorum) → `verify` → `settle` | `refund`. `JobKind = {Inference, Training, Compute}`; modality/tier/params live in a content-addressed spec at `Job.spec_hash`. Settlement uses **one canonical ComputeReceipt** at a fixed **8 PYRX per Compute Unit** (metered in CU-milli); escrow supports drip-release and multi-party `release_split`.

Verification ladder (all built): (1) **redundant exact-hash quorum within a pinned KernelClass** (cross-GPU inference isn't bit-reproducible, so agreement is required only within one kernel class); (2) **optimistic fraud proofs** with a challenge window; (3) an **interactive bisection dispute** (a Merkle trace commitment halved over $\lceil \log_2 N \rceil$ rounds to the single divergent step, where the honest defender always wins); (4) **TEE attestation** gating the Trusted tier. **GPU cohort pooling** combines partial results with a **member-independent SHA-256** so a Python sidecar reproduces it byte-for-byte and two honest cohorts of different members agree.

Trust tiers: **Open** (open-lineage weights, any consumer GPU) < **Secure/Sharded** (open models sharded across consumer GPUs — deterrence, not cryptographic proof) < **Trusted** (TEE-attested datacenter GPUs only; proprietary weights, sealed jobs). A Trusted model on a consumer node hard-routes to the network.

Local-first. `pyrax-local-runtime` is a deterministic, GPU-free router whose **RTX 3060 (12 GB) = Baseline** target; its `decide_route` **delegates to the real network scheduler**, so "run local" is provably the decision the network would make. The `neurax-route` node command is shared by the desktop app's NEURAX tab and the CLI. Payment integrity is hardened: payout is derived from a **verified secp256k1 pubkey** (never a self-declared worker string), with per-shard CU clamping and collision-safe canonical spec encodings.

Economics (§14): the **4B AI-Compute pool** funds a ~70M PYRX/month budget, transitioning to revenue (job fees + the treasury gas share) as coverage rises, after which the unspent pool returns to the DAO. The program is explicit about physics: consumer video is minutes-not-realtime; WAN-sharded 100B+ inference is batch 1–5 tok/s; ZK-ML cannot verify a 7–70B forward pass in 2026 (hence the verification ladder, not ZK-ML); WAN consumer training has no compute-integrity proof (stake + reputation + audit only). Deferred: real TEE quote-chains, the NEURAX gateway/SDK, the copilot, the 9.7 video runtime.

12. Crucible — Zero-Fee First-Party Mining

`pyrax-crucible` is a NiceHash-class miner-manager with a **hard-coded zero developer fee**, pointed at the operator's **local** node (coinbase = the operator's own address; no third-party pool, no hidden routing). Its lane→hardware→algo→backend matrix drives the five TriStream lanes: A (BLAKE3 + SHA-256d) → ASIC Stratum; B (kHeavyHash) → GPU; B (Argon2id) → CPU; Stream C is staking-status only. By construction a Crucible solution is accepted by the node's pure-Rust verifiers. **Status: design-complete scaffold; the full build is deferred until the L1 mainnet program passes.**

13. NOVA — Autonomous Operations

`pyrax-nova` (`status.pyraxnetwork.org`, **built + live**) is the ecosystem's observability and autonomous ops brain: **patrol** → **dedup** → **investigate** → **dossier** → **RAG-assess** → **dispatch** → **edge auto-repair [test-gated]** → **monitor** → **resolve** → **learn**. An uptime engine checks every surface (including a `chain` monitor that flags STALLED if height stops advancing) behind an SSRF guard; faults are routed to the repo that *owns* them; an edge box drives a headless coding agent against an allow-listed checkout with a **diff-level red-line file guard** and **runs the full test suite, shipping only on green** (auto-rollback otherwise). It is **advisory-first** (autonomy off by default; chain-service restarts are always human-approved; a 15-minute warning broadcasts before any chain-affecting action), with two red-line guards and an append-only action audit. The reasoning model runs locally on the operator's GPU.

14. Tokenomics

(All figures below are enforced or defined in code; the SSOT is `TOKENOMICS.md` cross-checked against `pyrax-consensus`, `pyrax-state`, and the `pyrax-contracts` crate.)

14.1 The token

Field	Value
Name / ticker	PYRAX / PYRX
Decimals	18 (1 PYRX = 10^{18} base units — "ash"); EVM wei ↔ base unit 1:1
Max supply	50,000,000,000 PYRX — a hard cap enforced in consensus (<code>assert_genesis_supply</code> on every production net; waived only on the play-money Seed)
Composition	37.5B premine + 12.5B mined = 50B
Genesis price	\$0.0025 / PYRX (~\$125M fully-diluted)

14.2 Genesis distribution (the premine)

Seeded at genesis into reserved, credit-only **system accounts**:

Pool	Amount	% of 50B	Account
Genesis (public) distribution	25,000,000,000	50%	0x...0204
Ecosystem & liquidity	5,000,000,000	10%	0x...0205
AI-Compute & inference pool	4,000,000,000	8%	0x...0203
Team & advisors	2,500,000,000	5%	0x...0206
DAO treasury & reserve	1,000,000,000	2%	0x...0201
Premine total	37,500,000,000	75%	—
Mining emissions (minted to coinbase over ~26 yr)	12,500,000,000	25%	producer
PYRAX treasury (fee-funded)	0 at genesis	—	0x...0200

Genesis event: 20B PYRX sold at \$0.0025 = **\$50,000,000**, plus a **25% utility bonus (5B PYRX)** — so genesis participants receive **25B total** (the whole 0x...0204 pool). The bonus is framed as network access / compute credits, **never an investment return**.

Vesting (documented policy; the on-chain vesting contract is a later phase — the pools currently sit in credit-only accounts): genesis 100% at TGE; mining ~26 yr; ecosystem 40% at TGE + 60% linear over 24 months; AI-compute streamed over 48 months; team **12-month cliff then 36-month linear**; DAO 10% at TGE. **Initial circulating supply ≈ 27.1B (~54%).**

14.3 Emissions

`INITIAL_BLOCK_SUBSIDY = 300 PYRX` ; `subsidy(h) = 300 >> (h / 21,000,000)` clamped to the remaining cap; **halving every 21,000,000 blocks (~4 years at 6 s)**; **mining cap 12.5B** (fees only thereafter); ~26 years to the cap, ~80% in the first ~8 years. The **full subsidy is minted to the block producer's coinbase**; the even one-third-per-stream split is **emergent** (each stream produces $\approx \frac{1}{3}$ of blocks), not applied per block.

The genesis block mints nothing. A testnet-only ramp raises the reward from 5% to 100% over 864,000 blocks.

14.4 Fees (EIP-1559, consensus-frozen)

- **Base fee: 25% burned · 50% → PYRAX treasury · 25% → DAO.**
- **Priority tip: 70% → block producer · 20% → PYRAX treasury · 10% → DAO.**
- Transparent transfer intrinsic gas 21,000; the flat **shielded fee (100 base units) is burned.**
- The PYRAX-treasury share funds AI-compute payouts when the network is revenue-positive.

14.5 AI-compute economics

1 CU = 1 reference-GPU-hour (RTX-4090-class); hardware multipliers are measured (H100/A100 2.5–4.0×; RTX 4090 = 1.0; RTX 3060/4060 = 0.35×; CPUs 0.04–0.15×). **Base rate** `PYRX_PER_CU = 8` (metered in CU-milli); **~70M PYRX/month** pool budget (4B ÷ ~57 months); quality multiplier 0.8–1.2×; requester spot premium up to 2×; per-job pool drawdown cap 10,000 PYRX (1,250 CU). On-chain escrow (`Tx::Escrow`) with drip/split settlement. **Funding transition:** bootstrap from the 4B pool → as trailing-30-day revenue coverage rises 0→1, the pool's share of each payout tapers; at coverage ≥ 1.0 for 3 months the pool is fully replaced by revenue and the unspent balance **returns to the DAO treasury**. At the \$0.0025 genesis price, 8 PYRX/CU ≈ \$0.02/GPU-hour — the program is explicit that early provider margin depends on token appreciation.

14.6 Staking rewards

Validators earn the Stream-C emission share, the 70% producer tip on blocks they produce, and staking rewards; APR/commission/delegation curves are deferred to the emission workstream. Minimum stake **32 PYRX**, ~7-day unbonding, **5% equivocation slash + 10% reporter bounty**.

15. Governance & the DAO

On-chain governance (`Tx::Gov`) runs **propose → VOTING → automatic tally at the deadline → PASSED → automatic enact at the activation height**. Exactly **three parameters are governable**: `block_gas_limit` (ceiling 64×), `min_validator_stake` (1024×), `unbonding_period` (64×). A proposal locks a **1,000 PYRX deposit** (burned if quorum is not reached, refunded otherwise); any account may propose; any bonded validator votes, **stake-weighted** (latest vote wins). **Quorum ≥ 1/3** of bonded stake; **pass > 2/3** of voting stake. **Frozen forever — no vote can touch:** the producer-forward fee split, the 12.5B emission cap + halving, and the governance rules themselves. The DAO treasury (0x...0201) holds 1B at genesis (10% liquid) and accrues the base-fee 25% + tip 10% ongoing; **treasury-spending governance is a later layer**.

16. The Four Networks

Brand	slug	Chain ID	Mode	Block	Role
PYRAX Seed	seed	881,109	Simulated	5 s	permanent developer sandbox; ~1T play-money, cap-waived; 1M welcome faucet
PYRAX Forge	forge	710,823	Production	5 s	closed public alpha; 3 seeded genesis validators (0.1B each)
PYRAX Rise	rise	104,928	Production	6 s	official public testnet; testnet emission ramp
PYRAX One	one	563,821	Production	6 s	mainnet; genesis ceremony pending; no faucet, no dev keys

Genesis hashes are BLAKE3-pinned. The binary ships **exactly these four chainspecs**; a production chain-id can never fake instant-seal. Seed is a byte-identical simulation that stays a sandbox forever; Forge/Rise/One share the real Production consensus, differing only by chainspec, launch order, and audit gating.

17. Wallet, Node Apps, CLI, and SDK

- **Desktop node apps** (Electron): **Inferno** (public operator variant — full node, mines the streams, stakes, contributes AI/compute; no seeding) and **Ember** (internal seed variant). The renderer is presentational; all privileged actions cross a narrow, typed IPC (contextIsolation on, nodeIntegration off). The app supervises the compiled node binary and 3-stream miner via sidecars; the **CLI embeds the full node in-process**.
 - **Wallet security**: the vault uses **script ($N = 2^{17}$) + AES-256-GCM + OS-keychain seal + TOTP 2FA, fail-closed**. Keys never leave the main process; the **shielded spending key never reaches the node** (the node only receives the public shielded chain data needed for wallet-side proving). Derivation is BIP-44 coin type **7777** with per-network isolation. A browser/WASM prover (`pyrax-prover-wasm`) is scaffolded and audit-gated.
 - **SDK**: `pyrax-sdk` is a real working client (offline build/sign/submit + typed methods + a generic `call()` escape hatch). Node metrics are exported for Prometheus.
-

18. RPC Surface

The node exposes **35** `eth_*` methods (`chainId`, `blockNumber`, `getBalance`, `getTransactionCount`, `getCode`, `getStorageAt`, **`getProof`**, `getBlockBy*`, `getBlockReceipts`, `getTransactionBy*/Receipt`, `getLogs` [10k-block cap], **`call/estimateGas`**, `gasPrice/maxPriorityFeePerGas/feeHistory`, **`sendRawTransaction`**, `syncing`), **6 filter methods**, and `eth_subscribe` (`newHeads` / `logs` / `newPendingTransactions` over `WebSocket`); plus `net_*` / `web3_*`; plus **27 native** `pyrax_*` methods (`blockNumber`, **`syncStatus`** [honest

height/target/targetKnown], getBalance/nonce/getBlock, **noteState** and **shieldedChainData** [wallet-side proving; no keys to the node], **snapshot**, **consensusInfo**, dagTips/dagRecent, peerCount/peers/nodeInfo/bandwidth, sendTransaction, **getWork/submitWork** [the external-miner PoW on-ramp], the mixnet `stream*` calls, dial/dropPeer, token-gated seedKeygen/seedListSign, and subscribeNewHeads). A public node runs `--rpc-public` : a strict read-only allowlist enforced at the parsed-JSON-RPC layer that can never mutate state, with DoS clamps throughout.

19. Cryptographic Primitives

All implemented and tested in `pyrax-crypto` : **BLAKE3** (default hash), **SHA-256** (Stream A + interop), **Keccak-256** (EVM addresses); **secp256k1** (65-byte recoverable signatures, **low-s enforced** for canonicity; `recover_evm` accepts high-s for ECRECOVER parity; address = last 20 bytes of `keccak256(pubkey)`); **Ed25519** (node/P2P + seed-list publisher identity); **BLS12-381 (blst, min-pubkey)** with mandatory **proofs of possession** and subgroup/infinity key validation; **ChaCha20-Poly1305** AEAD (wallet vault + note encryption) and **X25519** ECDH (note encryption by address, low-order-point rejection).

20. Security, Threat Model, and the Audit Gate

PYRAX maintains a formal **invariant catalog** (each with an enforcement site + test): **M1–M4** monetary (supply $\leq 50B$, per-tx value conservation, frozen fee split, capped subsidy); **C1–C7** consensus (unforgeable difficulty, state-root binds execution, **finality overrides work**, real 2/3 BLS supermajority, no honest equivocation, governed gas limit, cross-lane blue-work); **Z1–Z4** privacy (real ZK verification by default, no double-spend, shielded value conservation, real L1 batch verification at depth 32); **G1–G4** governance; **I1–I4** integrity (replay protection, never-panic decoders, body integrity, authenticated senders). The **threat model** (A1–A8: malicious peer, Byzantine producer, minority staker/miner, stake-grinder, governance griever, resource exhaustion, privacy breaker, key thief) maps each capability to its mitigation; the BFT safety bound is $< 1/3$ Byzantine stake.

Threat	Defense
Consensus takeover (51%)	Three uncorrelated resources (ASIC PoW, GPU+CPU PoW, staked PoS); GhostDAG reds withheld branches; BLS finality makes reverting slashable.
Equivocation / nothing-at-stake	5% slash + 10% bounty; POP-protected, subgroup-validated aggregate attestations.
Shielded double-spend	Global nullifier set; any repeat rejected.
Forged shielded value	No-trusted-setup plonky2 proof + binding signature; pinned circuit digests; external circuit audit before mainnet.
Graph / network deanonymization	Shielded-by-default (no sender/receiver/amount) + Sphinx mixnet (fixed-size, content-blind, cover traffic).
Eclipse / spam / Sybil	Bootstrapless multi-source discovery, GossipSub scoring, per-/24 eclipse cap, ban scoring, replay protection.
Malicious compute worker	Sandboxed execution + the 4-rung verification ladder (redundancy → fraud proof → bisection → TEE).
Signature malleability	secp256k1 low-s enforcement — one authorization, one encoding.

The audit gate. The road to **One** is explicitly gated on **an external audit of the consensus design, the ZK circuits, and the bridge soundness** — the single external gate before mainnet is deployed. No shielded circuit ships to mainnet unaudited; the GhostDAG parameters and finality rules receive external review; an incentivized public testnet (Rise) and a bug bounty precede genesis. The system never self-deploys — deployment is a deliberate human action.

Honest limitations (documented, not hidden): the L3 sequencer is deferred (live bridge rejects all); mixnet cover-traffic emission and bandwidth-weighted routing are not yet built; sync currently gates on blue-score only; `pyrax-contracts`, `pyrax-prover-wasm`, and Crucible are stubs/scaffolds; the desktop OTA has no in-app artifact-signature verification yet (HTTPS + OS code-signing only); the CLI keystore KDF is not memory-hard (weaker than the desktop wallet's scrypt).

21. Implementation Status & Roadmap

What is real, tested code today: the GhostDAG; the 5-lane TriStream with per-lane DAA; BLS PoS finality with real staking + slashing; the Merkle-Patricia state with `eth_getProof` and fast-sync; the three VMs with cross-VM calls, precompiles, and the 6 tx envelopes; the plonky2 shielded-transfer prover (default-on, pinned digests); the L3 validity circuit + recursion + on-L1 verifier; the bootstrapless libp2p mesh with eclipse/ban control; the Sphinx mixnet; content-addressed file transfer + media streaming; the full `eth_*` / `pyrax_*` RPC + SDK; the NEURAX compute market (job lifecycle, escrow, 4-rung verification, GPU pooling, local router); the desktop apps, CLI, wallet, tunnel, sync worker, and directory; the five websites and their deploy bundle; and NOVA (live).

The **Real-Consensus** → **Mainnet program (P0–P13)** turns this faithful simulation into a secured, decentralized L1:

Phase	Theme	Status
P0	Cluster harness + 50B supply invariant + <code>ConsensusMode::Production</code> spine	Done
P1	Merkle-Patricia state (storage-in-root)	Done
P2	State proofs + <code>eth_getProof</code>	Done
P3	Fast-sync via finalized snapshot	Done (live-mesh auto-trigger residual)
P4	Real BLS PoS finality + staking economy	Done
P5	Real 5-lane TriStream (BLAKE3 + SHA-256d + kHeavyHash + Argon2id + PoS)	Done
P6	P2P hardening + binary codec (Sybil/eclipse resistance)	Done
P7	L3 recursive ZK rollup → 500k-TPS path	Verifier + prover done; sequencer go-live deferred
P8	ZK privacy default-on for production nets	Done
P9	Live-network CI + fuzzing + property/invariant tests	Done
P10	Real SDK + Prometheus metrics	Done
P11	Governance + validator-key management + release integrity	Partial (on-chain governance + double-sign guard + signed releases done; remote signer remaining)
P12	Forge closed-alpha standup	In progress
P13	Mainnet (One) readiness + external audit package	Not started

NEURAX (9.x) and Crucible advance on their own tracks behind their own audit gates; the L3 sequencer go-live and the external L1 audit are the principal items between here and Mainnet One.

22. Governance Philosophy & Licensing

Open-core. The protocol — node, consensus, ZK circuits, CLI, SDK, and smart-contract interfaces — is **Apache-2.0** (SPDX on the first line of every file). A decentralized chain's validity rules *must* be public for the network to be trustless. The **apps, wallet, NEURAX, and hosted services are proprietary** (`LicenseRef-PYRAX-Proprietary`). A restrictive code license would not protect a chain — any chain can be forked regardless of license — so PYRAX's defensible moat is the **brand (PYRAX™, a trademark)**, the **network and its effects**, the **wallet/app/NEURAX UX**, and the **off-chain services**, not source secrecy. Apache-2.0's explicit patent grant signals PYRAX will not patent-troll its users or integrators.

100%-decentralization stance. No project-run infrastructure sits on the critical path: discovery is bootstrapless, relays are content-blind, the seed list is a signed community artifact, and the explorer must respect privacy and never deanonymize shielded activity. The network can run, and users can join, without trusting or depending on any single operator — including the PYRAX project itself.

23. References

1. Y. Sompolinsky, S. Wyborski, A. Zohar. *PHANTOM and GhostDAG: A Scalable Generalization of Nakamoto Consensus*.
2. The Kaspas project. *GhostDAG / k-cluster blue-set ordering; kHeavyHash*.
3. E. Ben-Sasson, A. Chiesa, et al. *Zerocash — the note-commitment / nullifier model*.
4. The Electric Coin Company / Zcash. *Orchard* shielded protocol.
5. S. Bowe, J. Grigg, D. Hopwood. *Recursive Proof Composition without a Trusted Setup (Halo)*.
6. Polygon Zero. *Plonky2: Fast Recursive Arguments with PLONK and FRI*.
7. G. Danezis, I. Goldberg. *Sphinx: A Compact and Provably Secure Mix Format*.
8. Nym / Loopix. *Anonymous communication via stratified mixnets with cover traffic*.
9. P. Maymounkov, D. Mazières. *Kademlia*.
10. Protocol Labs. *libp2p*.
11. J. O'Connor, J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn. *BLAKE3*.
12. D. Boneh, M. Drijvers, G. Neven. *Compact Multi-Signatures — BLS aggregation and proofs of possession*.
13. tevador. *Argon2id / RandomX* memory-hard proof-of-work.
14. Apache Software Foundation. *Apache License, Version 2.0*.

PYRAX™ — high-throughput, private-by-default, fully decentralized, ISP-resistant, and open-core. This whitepaper describes the protocol as implemented as of 2026-06-30. Consensus parameters, circuit details, and economic constants are code-enforced today and subject to external audit before Mainnet One; figures herein are code-verified, and the honest limitations of the current build are stated in §20–§21.